

Autodesk® Scaleform®

Scaleform 游戏通讯概述

Scaleform 3.1

C++ Flash ActionScript

Mustafa Thamer

2.01

2010 9 21

Copyright Notice

Autodesk® Scaleform® 4.4

© 2014 Autodesk, Inc. All rights reserved. Except as otherwise permitted by Autodesk, Inc., this publication, or parts thereof, may not be reproduced in any form, by any method, for any purpose.

Certain materials included in this publication are reprinted with the permission of the copyright holder.

The following are registered trademarks or trademarks of Autodesk, Inc., and/or its subsidiaries and/or affiliates in the USA and other countries: 123D, 3ds Max, Algor, Alias, AliasStudio, ATC, AutoCAD LT, AutoCAD, Autodesk, the Autodesk logo, Autodesk 123D, Autodesk CAM 360, Autodesk Homestyler, Autodesk Inventor, Autodesk MapGuide, Autodesk Streamline, AutoLISP, AutoSketch, AutoSnap, AutoTrack, Backburner, Backdraft, Beast, BIM 360, Burn, Buzzsaw, CADmep, CAiCE, CAMduct, CFdesign, Civil 3D, Cleaner, Combustion, Communication Specification, Configurator 360™, Constructware, Content Explorer, Creative Bridge, Dancing Baby (image), DesignCenter, DesignKids, DesignStudio, Discreet, DWF, DWG, DWG (design/logo), DWG Extreme, DWG TrueConvert, DWG TrueView, DWGX, DXF, Ecotect, ESTmep, Evolver, FABmep, Face Robot, FBX, Fempro, Fire, Flame, Flare, Flint, FMDesktop, ForceEffect, FormIt, Freewheel, Fusion 360, Glue, Green Building Studio, Heidi, Homestyler, HumanIK, i-drop, ImageModeler, Incinerator, Inferno, InfraWorks, InfraWorks 360, Instructables, Instructables (stylized robot design/logo), Inventor, Inventor HSM, Inventor LT, Kynapse, Kynogon, LandXplorer, Lustre, MatchMover, Maya, Maya LT, Mechanical Desktop, MIMI, Mockup 360, Moldflow Plastics Advisers, Moldflow Plastics Insight, Moldflow, Moondust, MotionBuilder, Movimento, MPA (design/logo), MPA, MPI (design/logo), MPX (design/logo), MPX, Mudbox, Navisworks, ObjectARX, ObjectDBX, Opticore, Pipeplus, Pixlr, Pixlr-o-matic, Productstream, Publisher 360, RasterDWG, RealDWG, ReCap, ReCap 360, Remote, Revit LT, Revit, RiverCAD, Robot, Scaleform, Showcase, Showcase 360 ShowMotion, Sim 360, SketchBook, Smoke, Socialcam, Softimage, Sparks, SteeringWheels, Stitcher, Stone, StormNET, TinkerBox, ToolClip, Topobase, Toxik, TrustedDWG, T-Splines, ViewCube, Visual LISP, Visual, VRED, Wire, Wiretap, WiretapCentral, XSI.

All other brand names, product names or trademarks belong to their respective holders.

Disclaimer

THIS PUBLICATION AND THE INFORMATION CONTAINED HEREIN IS MADE AVAILABLE BY AUTODESK, INC. "AS IS." AUTODESK, INC. DISCLAIMS ALL WARRANTIES, EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE REGARDING THESE MATERIALS.

Scaleform

MD 20770

6305 310

Scaleform

www.scaleform.com

info@scaleform.com

(301) 446-3200

(301) 446-3199

1	C++ Flash ActionScript	1
1.1	ActionScript C++	2
1.1.1	FSCCommand	2
1.1.2	API	3
1.2	C++ ActionScript	7
1.2.1	ActionScript	7
1.2.2	ActionScript	8
1.2.3		10
2	Direct Access API	12
2.1		13
2.2		14
2.3	函数对象	14
2.4	Direct Access	17
2.4.1	Object	17
2.4.2	Array	18
2.4.3	显示对象支持	19
2.4.4	影片剪辑支持	19
2.4.5	文本字段支持	20

1.1 ActionScript C++

Scaleform C++ ActionScript *FSCommand* *ExternalInterface*
ActionScript API Flash FSCommand
ExternalInterface Scaleform C++ Scaleform
GFX Gfx::Loader Gfx::MovieDef Gfx::Movie
[Scaleform documentation.](#)
FSCommand ActionScript 'fscommand' fscommand ActionScript
C++ fscommand C++
fscommand ActionScript
ExternalInterface ActionScript *flash.external.ExternalInterface.call*
ActionScript C++ ExternalInterface
Gfx::Value ActionScript Gfx::Value C++ ExternalInterface
Gfx::Value ActionScript ExternalInterface.call ActionScript
ExternalInterface API Scaleform
ActionScript Flash
FSCommands ExternalInterface
Direct Access API Gfx::FunctionHandler FSCommands ExternalInterface
ActionScript VM C++ ActionScript
C++ Gfx::FunctionHandler [Direct](#)

[Access API](#)

1.1.1 FSCommand

ActionScript fscommand ActionScript

```
fscommand("setMode", "2");
```

fscommand

```

ActionScript fscommand      Gfx FSCommand
Gfx::FSCommandHandler      fscommand      Gfx::Loader
Gfx::MovieDef      Gfx::Movie
Gfx::Loader -> Gfx::MovieDef -> Gfx::Movie
      Gfx::RenderConfig      EdgeAA      Gfx::Translator

      Gfx::Loader      Gfx::Movie
      FSCommand      GfxPlayerTiny      (
"FxPlayerFSCommandHandler")

```

```

fscommand      Gfx::FSCommandHandler

```

```

class OurFSCommandHandler : public FSCommandHandler
{
    public:
    virtual void Callback(MovieView* pmovie, const char* pcommand,
        const char* parg)
    {
        Printf("FSCommand: %s, Args: %s", pcommand, parg);
    }
};

```

```

      ActionScript      fscommand
fscommand      fscommand

```

```

Gfx::Loader

```

```

// Register our fscommand handler
Ptr<FSCommandHandler> pcommandHandler = *new OurFSCommandHandler;
gfxLoader->SetFSCommandHandler(pcommandHandler);

```

```

Gfx::Loader      Gfx::Movie      Gfx::MovieDef
SetFSCommandHandler

```

```

      C++
Advance      Invoke calls

```

1.1.2 API

Flash ExternalInterface.

fscommand

ExternalInterface

fscommand

ExternalInterface

```
class OurExternalInterfaceHandler : public ExternalInterface
{
public:
    virtual void Callback(MovieView* pmovieView, const char* methodName,
                        const Value* args, unsigned argCount)
    {
        Printf("ExternalInterface: %s, %d args: ", methodName, argCount);
        for(unsigned i = 0; i < argCount; i++)
        {
            switch(args[i].GetType())
            {
                case Value::VT_Number:
                    Printf("%3.3f", args[i].GetNumber());
                    break;
                case Value::VT_String:
                    Printf("%s", args[i].GetString());
                    break;
            }
            Printf("%s", (i == argCount - 1) ? "" : ", ");
        }
        Printf("\n");

        // return a value of 100
        Value retValue;
        retValue.SetNumber(100);
        pmovieView->SetExternalInterfaceRetVal(retValue);
    }
};
```

GFx::Loader

```
Ptr<GFxExternalInterface> pEIHandler = *new OurExternalInterfaceHandler;
gfxLoader.SetExternalInterface(pEIHandler);
```

ActionScript

ExternalInterface

1.1.2.1 FxDelegate


```

ExternalInterface
    methodName

Gfx::Value
Apps\Samples\GameDelegate\FxGameDelegate.h)

FxDelegate    Gfx::ExternalInterface    /
    methodName

FxDelegate    HUD Kit    FxDelegate    minimap
    minimap    FxDelegate

// Minimap Begin
void FxPlayerApp::Accept(FxDelegateHandler::CallbackProcessor* cbreg)
{
    cbreg->Process("registerMiniMapView", FxPlayerApp::RegisterMiniMapView);

    cbreg->Process("enableSimulation", FxPlayerApp::EnableSimulation);
    cbreg->Process("changeMode", FxPlayerApp::ChangeMode);
    cbreg->Process("captureStats", FxPlayerApp::CaptureStats);

    cbreg->Process("loadSettings", FxPlayerApp::LoadSettings);
    cbreg->Process("numFriendliesChange", FxPlayerApp::NumFriendliesChange);
    cbreg->Process("numEnemiesChange", FxPlayerApp::NumEnemiesChange);
    cbreg->Process("numFlagsChange", FxPlayerApp::NumFlagsChange);
    cbreg->Process("numObjectivesChange", FxPlayerApp::NumObjectivesChange);
    cbreg->Process("numWaypointsChange", FxPlayerApp::NumWaypointsChange);
    cbreg->Process("playerMovementChange", FxPlayerApp::PlayerMovementChange);
    cbreg->Process("botMovementChange", FxPlayerApp::BotMovementChange);

    cbreg->Process("showingUI", FxPlayerApp::ShowingUI);
}

FxDelegate

```

1.1.2.2 ActionScript

ActionScript

```

import flash.external.*;
ExternalInterface.call("foo", arg);

```

"foo"	"arg"	0
ExternalInterface API	Flash	

ExternalInterface	ExternalInterface
-------------------	-------------------

```
flash.external.ExternalInterface.call("foo", arg)
```

1.1.2.3 ExternalInterface.addCallback

ExternalInterface.addCallback	ActionScript	C++
		C++
Gfx::Movie::Invoke()		

AS Code:

```
import flash.external.*;

var txtField:TextField = this.createTextField("txtField",
                                             this.getNextHighestDepth(), 0, 0, 200, 50);

var aliasName:String = "setText";
var instance:Object = txtField;
var method:Function = SetText;
var wasSuccessful:Boolean = ExternalInterface.addCallback(aliasName, instance,
                                                         method);

function SetText()
{
    trace(this + ".SetText ");
    this.text = "INVOKED!";
}
```

SetText ActionScript	"this"	txtField
----------------------	--------	----------

C++ Code:

```
pMovie->Invoke("setText", "");
```

"txtField.SetText"	"INVOKED!"	"txtField"
Gfx::Movie::Invoke()	1.2.2	.

1.2 C++ *ActionScript*

ActionScript C++ Scaleform C++ Flash
 Scaleform C++ ActionScript
 Direct Access API C++ ActionScript
[Direct Access API](#)

1.2.1 *ActionScript*

Scaleform [GetVariable](#) and [SetVariable](#), ActionScript
 Direct Access API 2
 Set/GetVariable Direct Access API
 Direct Access call Gfx::Value::SetMember
 Gfx::Value GetVariable ActionScript
 Direct Access

GetVariable SetVariable ActionScript

```
int counter = (int)pHUDMovie ->GetVariableDouble("_root.counter");
counter++;
pHUDMovie->SetVariable("_root.counter", Value((double)counter));
```

GetVariableDouble _root.counter C++ Double
 GetVariableDouble 0. SetVariable
 _root.counter [Gfx::Movie](#) GetVariable SetVariable
 SetVariable Gfx::Movie::SetVarType " "
 Flash t _root.mytextfield
 SetVariable("_root.mytextfield.text", "testing", SV_Normal) 1
 SV_Sticky ()
 _root.mytextfield.text 3
 C++ SV_Normal SV_Sticky
 SV_Normal
 Scaleform [SetVariableArray](#) [GetVariableArray](#)

SetVariableArray

Gfx::Movie::SetVariableArraySize

SetVariableArray AS

```
int idxInASArray = 0;
const char* strarr[2];
strarr[0] = "This is the first string";
strarr[1] = "This is the second one";
pMovie->SetVariableArray(Movie::SA_String, "_root.strArray",
                        idxInASArray, strarr, 2);
```

```
int idxInASArray = 0;
const wchar_t* strarr[2];
strarr[0] = L"This is the first string";
strarr[1] = L"This is the second one";
pMovie->SetVariableArray(Movie::SA_StringW, "_root.strArray",
                        idxInASArray, strarr, 2)
```

The GetVariableArray AS

1.2.2 ActionScript

ActionScript

ActionScript

[Gfx::Movie::Invoke\(\)](#)

UI

UI

Direct Access API

ActionScript

_root level

"mySlider"

SetSliderPos

"mySlider"

AS Code:

"gripClip"	"mySlider"	pos	rangeMin/Max
------------	------------	-----	--------------

```
Value result;
bool bInvoked = pMovie->Invoke("_root.mySlider.SetSliderPos", &result, "%d",
                                newPos);
```

```
Value result;
if (pMovie->IsAvailable("parentPath.mySlider.SetSliderPos"))
    pMovie->Invoke("parentPath.mySlider.SetSliderPos", &result, "%d", newPos);
```

9

1.2.3

Flash

GfX::Movie

```
Movie::IsAvailable(path)
Movie::SetVariable(path, value)
Movie::SetVariableDouble(path, value)
Movie::SetVariableArray(path, index, data, count)
Movie::SetVariableArraySize(path, count)
Movie::GetVariable(value, path)
Movie::GetVariableDouble( path)
Movie::GetVariableArray(path, index, data, count)
Movie::GetVariableArraySize(path)
Movie::Invoke(pathToMethodName, result, argList, ...)
```

."

"clip2"

"clip1"

Main Stage -> clip1 -> clip2

1. clip1 /
2. clip2 clip1

```
"clip1.clip2"
"_root.clip1.clip2"
"_level0.clip1.clip2"
```

```
"Clip1.clip2" <- "Clip1"
"clip2" <- "clip1." -
```

"_root" "_level0" ActionScript 2

ActionScript 3

ActionScript 3

SWF

_root

o

stage

_root _level0 root level0

_levelN

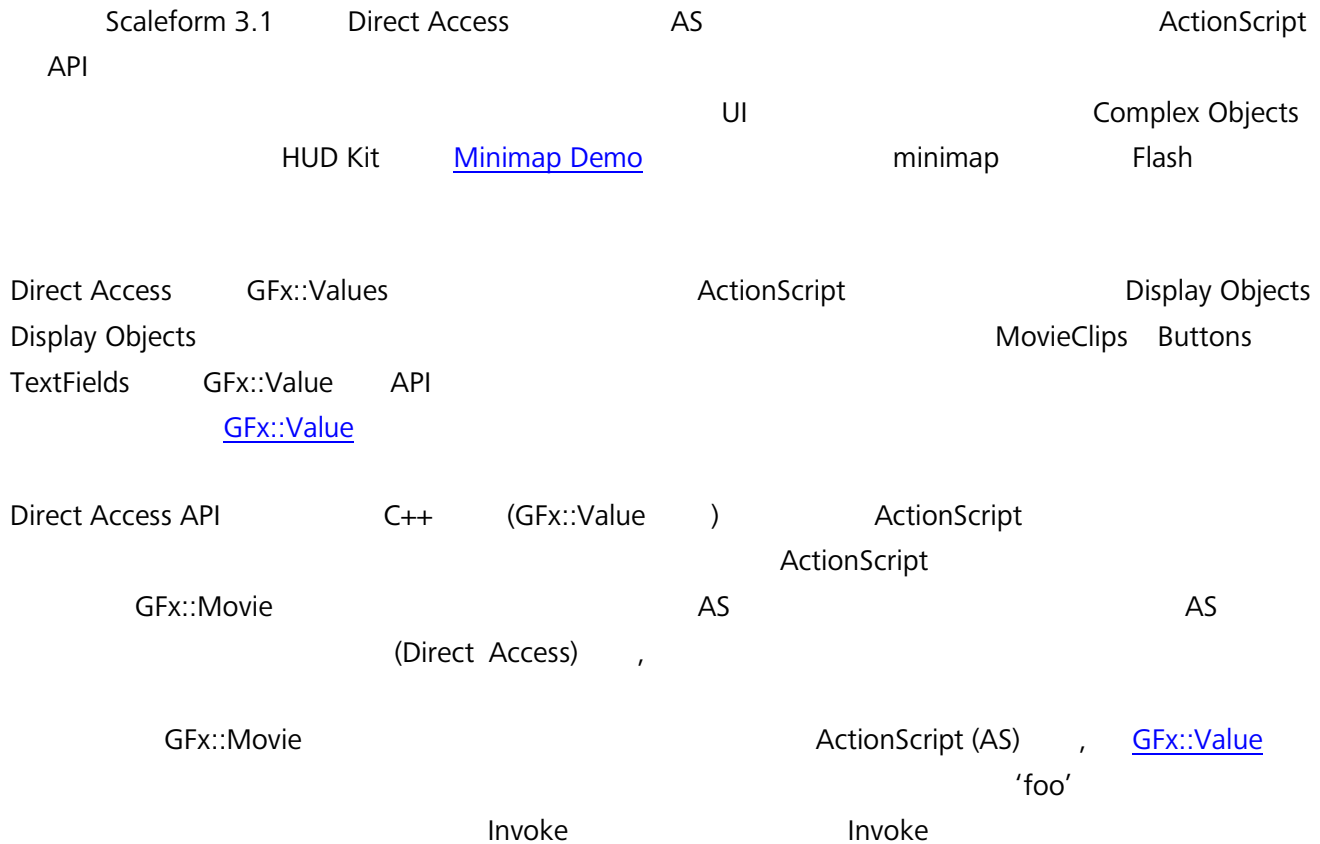
- Flash Studio Test Movie (Ctrl-Enter) (Ctrl-Alt-V)
()

- 1
(Ctrl-Alt-V) ()

- loadMovie
_level0 _level1 _level2 ()
_levelN.objectName
_levelN.objectPath.objectName N

1. [Paths to Objects and Variables](#) Jesse Stratford
2. [Advanced Pathing](#) Jesse Stratford

2 Direct Access API



1. Gfx::Movie Invoke ():

```
Value ret;
Value args[N];
pMovie->Invoke("_root.foo.method", &ret, args, N);
```

2. Direct Access Invoke ():

```
(Assumes 'foo' holds a reference to an AS object at "_root.foo")
Value ret;
Value args[N];
bool = foo.Invoke("method", &ret, args, N);
```

Invoke AS Direct Access API
 Gfx::Value::HasMember GetMember SetMember.
 GetMember MovieClip (Gfx::Value)


```
Value tf;
MovieClip.GetMember("textField", &tf);
tf.SetText("hello");
```

MovieClip

textField

SetText

2.1

Direct Access API

AS

C++

```
Movie* pMovie = ... ;
Value parent, child;
pMovie->CreateObject(&parent);
pMovie->CreateObject(&child);
parent.SetMember("child", child);
```

[CreateObject](#)

ActionScript

```
Value mat, params[6];
// (          params[0..6] ) ...
pMovie->CreateObject(&mat, "flash.geom.Matrix", params, 6);
```

```
// (    pMovie      Gfx::Movie* ):
Value owner, childArr, childObj;
pMovie->CreateArray(&owner);           //
pMovie->CreateArray(&childArr);        //
pMovie->CreateObject(&childObj);       //
bool = owner.SetElement(0, childArr); // parent[0]
bool = owner.SetElement(1, childObj); // parent[1]
...
bool = foo.SetMember("owner", owner); //          foo      'owner'
```

[Gfx::Value::VisitMembers\(\)](#)

ObjectVisitor

Visit

DisplayObject

VisitMembers

ActionScript

ASSetPropFlags

e.g.: `_global.ASSetPropFlags(MyClass.prototype, ["someFunc", "__get__number", "test"], 6, 1);`

Properties (getter/setter)

VisitMembers

ASSetPropFlags

Direct Access

VT_Object

"[]" VT_Object ()

"[]"

2.2

Direct Access API

Object

DisplayObject

MovieClips Buttons TextFields

[DisplayInfo](#) API

DisplayInfo API

DisplayObject

SetMember

SetDisplayInfo

Gfx::Movie::SetVariable

SetDisplayInfo

movieclip

```
// MovieClip Gfx::Value*
Value::DisplayInfo info;
PointF pt(100,100);
info.SetRotation(90);
info.SetPosition(pt.x, pt.y);
MovieClip.SetDisplayInfo(info);
```

2.3 函数对象

ActionScript2

(AS2 VM)

Gfx::Movie::CreateFunction

C++

CreateFunction

AS

C++

fscommand

ExternalInterface

AS

```

Value obj;
pmovie->GetVariable(&obj, "_root.obj");

class MyFunc : public FunctionHandler
{
    public:
        virtual void Call(const Params& params)
        {
            //      /
        }
};
Ptr<MyFunc> customFunc = *SF_HEAP_NEW(Memory::GetGlobalHeap()) MyFunc();
Value func;
pmovie->CreateFunction(&func, customFunc);
obj.SetMember("func", func);

```

 ActionScript (_root timeline)

```
obj.func(param1, param2, param3);
```

- | | | | |
|-----------|------------|---|--|
| Call() | Params | | |
| • pRetVal | Gfx::Value | Gfx::Value | |
| | | pMovie->CreateObject() or CreateArray() | |
-
- pMovie
 - pThis
 - ArgCount
 - pArgs Gfx::Values []
 - Gfx::Value firstArg = params.pArgs[0];
 - pArgsWithThisRef
 - pUserData

```

Value networkProto, networkCtorFn, networkConnectFn;
class NetworkClass : public FunctionHandler
{
    public:
        enum Method

```

```

{
    METHOD_Ctor,
    METHOD_Connet,
};

virtual ~NetworkClass() {}
virtual void Call(const Params& params)
{
    int method = int(params.pUserData);
    switch (method)
    {
    case METHOD_Ctor:
        //
        break;
    case METHOD_Connet:
        //
        break;
    }
}
};

Ptr<NetworkClass> networkClassDef = *SF_HEAP_NEW(Memory::GetGlobalHeap())
                                   NetworkClass();

//
pmovie->CreateFunction(&networkCtorFn, networkClassDef,
                    (void*)NetworkClass::METHOD_Ctor);

//
pmovie->CreateObject(&networkProto);
//
networkCtorFn.SetMember("prototype", networkProto);
// " "
pmovie->CreateFunction(&networkConnectFn, networkClassDef,
                    (void*)NetworkClass::METHOD_Connet);
networkProto.SetMember("connect", networkConnectFn);
// _global
pmovie->SetVariable("_global.Network", networkCtorFn);

```

```

var netObj:Object = new Network(param1, param2);

```

```

Value origFuncReal;
obj.GetMember("funcReal", &origFuncReal);

```

```

class FuncRealIntercept : public FunctionHandler
{
    Value OrigFunc;
public:
    FuncRealIntercept(Value origFunc) : OrigFunc(origFunc) {}
    virtual ~FuncRealIntercept() {}
    virtual void Call(const Params& params)
    {
        //
        OrigFunc.Invoke("call", params.pRetVal, params.pArgsWithThisRef,
                        params.ArgCount + 1);
        //
    }
};
Value funcRealIntercept;
Ptr<FuncRealIntercept> funcRealDef = *SF_HEAP_NEW(Memory::GetGlobalHeap())
                                     FuncRealIntercept(origFuncReal);
pmovie->CreateFunction(&funcRealIntercept, funcRealDef);
obj.SetMember("funcReal", funcRealIntercept);

Gfx::Value
.swf (Gfx::Movie)                                     Gfx::Values

```

2.4 Direct Access

Direct Access API

[Gfx::Value](#)

2.4.1 Object

	('foo' Object "_root.foo")
	bool has = foo.HasMember("bar");
	Value bar; bool = foo.GetMember("bar", &bar);
	Value val; bool = foo.SetMember("bar", val);
	Value ret; Value args[N]; bool = foo.Invoke("method", args, N, &ret); or foo.Invoke("method", args, N);

	Value obj; pMovie->CreateObject(&obj); ... bool = foo.SetMember("bar", obj);
	Value owner, child; pMovie->CreateObject(&owner); pMovie->CreateObject(&child); bool = owner.SetMember("child", child); ... bool = foo.SetMember("owner", owner);
	Value::ObjectVisitor v; foo.VisitMembers(&v);
	bool = foo.DeleteMember("bar");

2.4.2 Array

	('bar' Array "_root.foo.bar" 'foo' Object)
	unsigned sz = bar.GetArraySize();
	Value val; bool = bar.GetElement(idx, &val);
	Value val; bool = bar.SetElement(idx, val);
	bool = bar.SetArraySize(N);
	Value arr; pMovie->CreateArray(&arr); ... bool = foo.SetMember("bar", arr);
Complex Object	Value owner, childArr, childObj; pMovie->CreateArray(&owner); pMovie->CreateArray(&childArr); pMovie->CreateObject(&childObj); bool = owner.SetElement(0, childArr); bool = owner.SetElement(1, childObj); ... bool = foo.SetMember("owner", owner);
	for (UPInt i=0; i <N; i++) { ... bool = bar.GetElement(i+idx, &val) ... } : Value::ArrayVisitor v; bar.VisitElements(v, idx, N);

	<code>bool = bar.ClearElements();</code>
	<code>PushBack</code> <code>Value val;</code> <code>bool = bar.PushBack(val);</code> <code>PopBack</code> <code>Value val;</code> <code>bool = bar.PopBack(&val); or void bar.PopBack();</code>
	<code>bool = bar.RemoveElement(idx);</code> <code>bool = bar.RemoveElements(idx, N);</code>

2.4.3 显示对象支持

	<code>('foo' "_root.foo.bar" (MovieClip, TextField, Button)</code> <code>)</code>
	<code>Value::DisplayInfo info;</code> <code>bool = foo.GetDisplayInfo(&info);</code>
	<code>Value::DisplayInfo info;</code> <code>...</code> <code>bool = foo.SetDisplayInfo(info);</code>
	<code>Matrix2_3 mat;</code> <code>bool = foo.GetDisplayMatrix(&mat);</code>
	<code>Matrix2_3 mat;</code> <code>...</code> <code>bool = foo.SetDisplayMatrix(mat);</code>
	<code>Render::Cxform cxform;</code> <code>bool = foo.GetColorTransform(&cxform);</code>
	<code>Render::Cxform cxform;</code> <code>...</code> <code>bool = foo.SetColorTransform(cxform);</code>

2.4.4 影片剪辑支持

	<code>('foo' "_root.foo")</code>
	<code>Value newInstance;</code>

	<code>bool = foo.AttachMovie(&newInstance, "SymbolName", "instanceName");</code>
	<code>Value emptyInstance;</code> <code>bool = foo.CreateEmptyMovieClip(&emptyInstance, "instanceName");</code>
	<code>bool = foo.GotoAndPlay("myframe");</code>
	<code>bool = foo.GotoAndPlay(3);</code>
	<code>bool = foo.GotoAndStop("myframe");</code>
	<code>bool = foo.GotoAndStop(3);</code>

2.4.5 文本字段支持

	('foo' "_root.foo")
	<code>Value text;</code> <code>bool = foo.GetText(&text);</code>
	<code>Value text;</code> ... <code>bool = bar.SetText(text);</code>
HTML	<code>Value htmlText;</code> <code>bool = bar.GetTextHTML (&htmlText);</code>
HTML	<code>Value htmlText;</code> ... <code>bool = bar.SetTextHTML(htmlText);</code>